PYTHON PROGRAMMING

ASSIGNMENT 1

Design, Implement and Validate a Payment Calculator
for a Village Milk Co-operative

Student detail	Information
Name	S Jai Aakash
Register Number	1919260011
Course	Python Programming
Assignment	CO1 – Values, Types, Variables, Expressions, Functions and Modules
Total Marks	50
Submission Date	23 October 2025 (as stated in brief)
GitHub Repository	CSA08-1919260011-Assignment1 (link to be added after repository creation)

Prepared report: Tasks 1–12, source code, pseudocode, flowchart, test cases, complexity analysis and design-challenge evaluation.

Important implementation and validation notes

This report uses Decimal values constructed from strings for money and rate arithmetic. It applies ROUND_HALF_UP to the nearest paisa for each farmer payment. The original brief does not specify a rounding policy for half-paisa ties; this report explicitly adopts ROUND_HALF_UP and recommends confirming it with the co-operative's accounting policy.

The original rate formula is: base ■30.00 at fat 3.5% and SNF 8.5%; each 0.1 percentage-point fat step changes rate by ■0.40 and each 0.1 percentage-point SNF step changes rate by ■0.30. The design challenge uses two-decimal analyser readings. The revised implementation treats one step as 0.10 percentage points and rounds the number of steps to the nearest integer, with Decimal ROUND_HALF_UP.

Execution-time results depend on the computer and Python version. Therefore, this report supplies runnable benchmark code but does not invent measured timings. Run the benchmark locally and paste its output into the final submission. Likewise, create the required GitHub repository and insert its real URL; no repository is claimed to have been created here.

Contents

• Task 1 – Problem and data analysis

• Task 2 – Values, types and exact money

• Task 3 – Variables and constants

• Task 4 – Expressions and rounding

• Task 5 – Input validation

• Task 6 – Functions and scope

• Task 7 – Module design

• Task 8 – Algorithm and flowchart

• Task 9 – Implementation and test plan

• Task 10 – Expected versus executed results and benchmarking

• Task 11 – Final design recommendation

• Task 12 – Seasonal rates and volume bonus

• Appendix – Complete runnable reference implementation and GitHub structure

Task 1 – Understand the problem and identify the data (4 marks)

Value	Type	Reason / validation
farmer_id	str	An identifier may contain letters or leading zeros; do not perform arithmetic on it.
litres	Decimal	Milk quantity may be fractional; must be greater than zero.
fat_percent	Decimal	Analyser measurement; allowed range 2.00–7.00 inclusive.
snf_percent	Decimal	Analyser measurement; assumed valid range 6.00–10.00 inclusive pending co-operative confirmation.
shift	str / enum	Accept only MORNING or EVENING.
rate_per_litre	Decimal	Calculated from the rate card and rounded to two decimal places.
payment	Decimal	Litres × rate, rounded to the nearest paisa.
ten_day_total	Decimal	Sum of each farmer's rounded payment; the payable ledger total.
season / date	date or str	Select the applicable seasonal rate card; date boundaries must be explicit.

Named constants: BASE_RATE = Decimal('30.00'), FAT_REFERENCE = Decimal('3.5'), SNF_REFERENCE = Decimal('8.5'), FAT_STEP = Decimal('0.1'), SNF_STEP = Decimal('0.1'), FAT_STEP_VALUE = Decimal('0.40'), SNF_STEP_VALUE = Decimal('0.30'), MIN_FAT = Decimal('2.0'), MAX_FAT = Decimal('7.0'). Constants prevent rate revisions from being applied inconsistently.

Rules: reject malformed numbers; reject negative or zero litres; enforce allowed fat and SNF ranges; accept only known shifts; calculate with decimal arithmetic; round each farmer's payable amount to two decimals; preserve the same farmer/shift/date in

the ledger; do not silently clamp invalid readings.

Assumptions: one record represents one farmer's one collection; ten-day total is the sum of payments after per-payment rounding; no tax, deductions, penalties or quality rejection rules were supplied; the SNF range is a provisional validation assumption and must be confirmed by the co-operative.

Alternative concept and complexity

With separate variables such as litres1, fat1, snf1, and so on, 300 farmers require 900 measurement variables plus repeated calculation logic, making loops, validation and later expansion difficult. A list of records such as records = [(farmer_id, litres, fat, snf, shift), ...] supports a loop and consistent processing. For n records, processing is O(n) time. Storing all records is O(n) space; a running total alone is O(1) additional space. The record-based design scales from 300 to 150,000 entries without rewriting the algorithm.

Task 2 – Explore values and types in the interpreter (4 marks)

```
>>> type(10)
<class 'int'>
>>> type(10.0)
<class 'float'>
>>> type("10")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type([10, 8.5])
<class 'list'>
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> 7 % 2
1
>>> -7 // 2
-4
>>> 0.1 + 0.2 == 0.3
False
```

The / operator performs true division; // floors the quotient, so –7 // 2 is –4 rather than truncating toward zero; % returns the remainder. Binary floating-point cannot represent many decimal fractions exactly, so 0.1 + 0.2 is slightly different from 0.3.

Complete the Code – exact Decimal total

```
from decimal import Decimal

payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

print("float  :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3,
      Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

Expected output (the float representation may vary slightly by Python/platform):

```
float  : 33400.30000000061
Decimal: 33400.30
False True
```

Use Decimal(amount), where amount is the original string. Decimal(float(amount)) imports the float's binary approximation, so it may display a long decimal expansion and defeats the purpose of exact decimal input. For integer paise, convert ■33.40 to 3340 paise and ■0.10 to 10 paise; the exact total is 3,340,030 paise = ■33,400.30. Do not round after every float addition as a substitute for a clear money model.

Alternative concept and complexity

Method	Time for n amounts	Extra storage	Exact?	Comment
Decimal from string	O(n)	O(1) accumulator	Yes for decimal inputs, under chosen rounding policy	Recommended for readable financial arithmetic.
Integer paise	O(n)	O(1) accumulator	Yes when all values are in whole paise	Fast and exact; fractional-paisa rates need a defined conversion policy.
float + round repeatedly	O(n)	O(1) accumulator	Not guaranteed	Rounding may hide some errors but does not make binary float exact.

```
import sys
from decimal import Decimal
print(sys.getsizeof(3340), sys.getsizeof(33.40),
      sys.getsizeof(Decimal("33.40")))
```

The exact byte counts are implementation- and version-dependent; execute the snippet in the submission environment and record its output. Each addition is O(1) for ordinary fixed-size values, making the full total O(n).

Task 3 – Choose variables and name constants (3 marks)

```
from decimal import Decimal

BASE_RATE = Decimal("30.00")
FAT_REFERENCE = Decimal("3.5")
SNF_REFERENCE = Decimal("8.5")
FAT_STEP = Decimal("0.1")
SNF_STEP = Decimal("0.1")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
MIN_FAT = Decimal("2.0")
MAX_FAT = Decimal("7.0")

value = 10
value = "ten"  # legal: Python is dynamically typed
a, b = 2, 5
a, b = b, a    # simultaneous swap
```

Names may contain letters, digits and underscores but cannot start with a digit or be a keyword. Use snake_case for variables/functions and UPPER_CASE for constants. Python does not enforce constants; the convention communicates intent. Assigning an int then a str is legal because names refer to objects and are dynamically typed.

Alternative: magic numbers such as 0.40 embedded in k expressions require k edits during a revision, whereas named constants require one edit. A RATE_CARD dictionary improves grouping and extensibility. Runtime remains O(1) per lookup/calculation; memory for a small rate card is O(1) with respect to the number of farmer entries.

Task 4 – Write correct expressions (4 marks)

```
fat_steps = ((fat - FAT_REFERENCE) / FAT_STEP).quantize(
    Decimal("1"), rounding=ROUND_HALF_UP)
snf_steps = ((snf - SNF_REFERENCE) / SNF_STEP).quantize(
    Decimal("1"), rounding=ROUND_HALF_UP)
rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
rate = rate.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
amount = (litres * rate).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
```

Unexpected results: (4.2 - 3.5) / 0.1 may be 7.000000000000001 because float arithmetic is approximate; round(…, 0) can recover the intended step for ordinary readings. 0.1 + 0.2 == 0.3 is False for the same reason. int(-1.8) is −1 (truncate toward zero), while math.floor(-1.8) is −2. Python round() uses ties-to-even for exact halfway values; Decimal with ROUND_HALF_UP expresses the financial rule explicitly.

Operator precedence: exponentiation ** binds tightly and associates right-to-left (2 ** 3 ** 2 is 512); unary minus has a subtle relationship to exponentiation (-2 ** 2 is -4, while (-2) ** 2 is 4). Use parentheses to make the intended order obvious. Comparison chaining 2.0 <= fat <= 7.0 is equivalent to two comparisons joined by and.

Truncation versus floor versus round

fat	Exact signed step	round to nearest	int truncation	floor
4.2	7	7	6	7
3.1	−4	−4	−3	−4

For fat 4.2, truncating the binary float result can yield 6 steps; for 3.1 it can yield –3 rather than –4. Floor happens to match the intended result for these values but is wrong for positive values with a tiny representation error above an integer, and it is not a general substitute for nearest-step rounding. Each calculation is O(1) time and O(1) space.

Task 5 – Read input and convert types (3 marks)

```
from decimal import Decimal, InvalidOperation

def read_decimal(prompt):
    text = input(prompt).strip()
    try:
        return Decimal(text)
    except InvalidOperation:
        raise ValueError("Enter a valid decimal, e.g. 8.5 (not 8,5 or eight).")

# Example formatting:
amount = Decimal("283.90")
print(f"Payment: Rs {amount:8.2f}")
```

input() always returns str. Convert IDs to str, whole-count fields to int and decimal measurements to Decimal. The string '8,5' is not a standard Decimal number and 'eight' is invalid; reject them with a useful message instead of guessing locale. Whitespace around ' 8.5 ' is removed by strip().

Alternative: float(text) in try/except ValueError is concise and catches '8,5' and 'eight', while accepting whitespace. It is still not the recommended representation for money. Both parsing approaches are O(1) per short entry; regular-expression/string pre-checks can reject malformed input but duplicate the numeric parser's job.

Task 6 – Organise the logic into functions (4 marks)

```
def rate_per_litre(fat, snf, rate_card):
    """Return the rounded price per litre for the supplied quality."""
    fat_steps = ((fat - rate_card["fat_reference"]) / rate_card["fat_step"]).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    snf_steps = ((snf - rate_card["snf_reference"]) / rate_card["snf_step"]).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    return (rate_card["base"] + fat_steps * rate_card["fat_value"]
            + snf_steps * rate_card["snf_value"]).quantize(
                Decimal("0.01"), rounding=ROUND_HALF_UP)

def payout(litres, fat, snf, rate_card):
    """Return one farmer's payment rounded to the nearest paisa."""
    return (litres * rate_per_litre(fat, snf, rate_card)).quantize(
        Decimal("0.01"), rounding=ROUND_HALF_UP)

def ten_day_total(payments):
    """Sum already-rounded farmer payments."""
    return sum(payments, Decimal("0.00"))
```

A function should return a value so the caller can test, reuse, format or store it; print() only displays text and returns None. Local variables exist within a function call. Constants/rate cards can be read, but changing global rates inside calculation functions would make behaviour harder to test. Docstrings explain each function's contract.

A long script can still run in O(n), but it tends to repeat logic and is harder to test. A lambda is suitable for a tiny expression, not a multi-step financial rule. Function-call overhead is O(1) per call; for n entries it remains O(n) and is normally negligible compared with I/O and Decimal arithmetic.

Task 7 – Build a module (4 marks)

```
# Save as milkrate.py
"""Rate card and calculation functions for the co-operative."""
from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
FAT_REFERENCE = Decimal("3.5")
SNF_REFERENCE = Decimal("8.5")
STEP = Decimal("0.1")

def rate_per_litre(fat, snf):
    """Return price per litre rounded to two decimal places."""
    fat_steps = ((fat - FAT_REFERENCE) / STEP).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    snf_steps = ((snf - SNF_REFERENCE) / STEP).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    return (BASE_RATE + fat_steps * FAT_STEP_VALUE
            + snf_steps * SNF_STEP_VALUE).quantize(Decimal("0.01"))

def payout(litres, fat, snf):
```

```
    """Return the farmer payment rounded to two decimal places."""
    return (litres * rate_per_litre(fat, snf)).quantize(
        Decimal("0.01"), rounding=ROUND_HALF_UP)

if __name__ == "__main__":
    for litres, fat, snf in [
        (Decimal("10"), Decimal("3.5"), Decimal("8.5")),
        (Decimal("8.5"), Decimal("4.2"), Decimal("8.7")),
        (Decimal("12"), Decimal("3.1"), Decimal("8.3"))]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(f"{litres:5} L fat {fat} snf {snf} rate {rate:6.2f} pay {pay:8.2f}")
```

Expected output:

```
 10 L fat 3.5 snf 8.5 rate  30.00 pay   300.00
8.5 L fat 4.2 snf 8.7 rate  33.40 pay   283.90
 12 L fat 3.1 snf 8.3 rate  27.80 pay   333.60
```

```
# Save as test_milkrate.py in the same folder
from decimal import Decimal
import milkrate as mr

print(mr.payout(Decimal("8.5"), Decimal("4.2"), Decimal("8.7")))
```

import milkrate exposes names through the module; from milkrate import payout imports that name directly; import milkrate as mr uses an alias. The if __name__ == '__main__' block runs only when the file is executed as the main program. During import, __name__ is 'milkrate', so the demonstration table does not print. Without the guard, import would run the demonstration unexpectedly. Python searches the current/script directory and entries in sys.path; standard modules include decimal and math.

Alternative: copying functions into every script duplicates code and creates k update locations for a rate revision. A module centralises the rule. Python normally executes module initialization once per interpreter and caches the module in sys.modules; importing has a one-time cost, while subsequent imports reuse the cached module.

Task 8 – Plan the algorithm before coding (4 marks)

Problem-solving steps

• Understand: identify inputs, valid ranges, price formula and rounding rule.

• Plan: write pseudocode and flowchart; separate validation from calculation.

• Write: implement module functions and the collection loop.

• Test: verify normal, boundary, invalid and multi-entry totals; measure scale.

SIMATS-style pseudocode

```
ALGORITHM MilkPayment
INPUT rate card and ten-day collection entries
SET total ← 0.00
FOR each entry
    READ farmer_id, litres, fat, snf, shift
    IF any field is invalid OR litres <= 0 OR fat not in [2.0, 7.0]
        DISPLAY validation error
        CONTINUE to next entry
    END IF
    fat_steps ← ROUND_HALF_UP((fat - 3.5) / 0.1)
    snf_steps ← ROUND_HALF_UP((snf - 8.5) / 0.1)
    rate ← 30.00 + fat_steps × 0.40 + snf_steps × 0.30
    rate ← round rate to 2 decimal places
    payment ← round(litres × rate to 2 decimal places)
    RECORD farmer_id, rate, payment
    total ← total + payment
END FOR
DISPLAY all payments and total
END ALGORITHM
```

Flowchart – validate one entry

```
[Start]
   |
[Read ID, litres, fat, SNF, shift]
   |
{Numeric fields valid?} --No--> [Show error] --> [Next entry]
   | Yes
{litres > 0 and ranges valid?} --No--> [Show error] --> [Next entry]
   | Yes
[Compute steps and rate]
   |
[Compute rounded payment]
   |
[Record payment and add to total]
   |
```

```
[Next entry / End and display total]
```

Hand trace for three entries

Entry	Litres	Fat	SNF	Steps fat/SNF	Rate	Payment
A	10.00	3.5	8.5	0 / 0	■30.00	■300.00
B	8.5	4.2	8.7	7 / 2	■33.40	■283.90
C	12.0	3.1	8.3	–4 / –2	■27.80	■333.60
Total						■917.50

Alternative: store all entries then sum them (O(n) time and O(n) storage), or update a running total after each accepted entry (O(n) time and O(1) extra storage). Store entries when an audit trail, correction, reprint, reconciliation or later analysis is required.

Task 9 – Implement and test the calculator (4 marks)

The reference implementation is included in the appendix. The following table gives deterministic example cases. Invalid-input behaviour is defined by the validation function; run the test suite to record actual output in your environment.

Case	Input	Expected result	Purpose
T1	10 L, fat 3.5, SNF 8.5	■300.00	Reference rate
T2	8.5 L, fat 4.2, SNF 8.7	■283.90	Above reference
T3	12 L, fat 3.1, SNF 8.3	■333.60	Below reference
T4	1 L, fat 2.0, SNF 8.5	■24.00	Minimum fat
T5	1 L, fat 7.0, SNF 8.5	■44.00	Maximum fat
T6	–1 L, fat 3.5, SNF 8.5	ValueError	Negative quantity
T7	1 L, fat 7.1, SNF 8.5	ValueError	Fat out of range
T8	litres='eight'	ValueError / parse error	Invalid text
T9	1 L, fat 3.5, SNF 8.5	■30.00	Boundary reference

Five-entry hand total example: 300.00 + 283.90 + 333.60 + 24.00 + 44.00 = ■985.50. The five test entries are illustrative and should be run through the submitted program.

```
def run_tests():
    from decimal import Decimal as D
    assert payout(D("10"), D("3.5"), D("8.5")) == D("300.00")
    assert payout(D("8.5"), D("4.2"), D("8.7")) == D("283.90")
    assert payout(D("12"), D("3.1"), D("8.3")) == D("333.60")
    assert payout(D("1"), D("2.0"), D("8.5")) == D("24.00")
    assert payout(D("1"), D("7.0"), D("8.5")) == D("44.00")
    print("All valid-payment assertions passed.")
```

Manual tests require retyping cases after changes and may miss a case. Assertions are repeatable and can be run after every edit; the test suite's runtime is O(m) for m tests. Automated tests reduce repeated human effort and are easier to reproduce.

Task 10 – Compare expectations with execution (4 marks)

For N identical payments of ■33.40 plus three ■0.10 adjustments, the exact Decimal total is N × 3340 paise + 30 paise. For N=300, 6,000 and 150,000 base payments, the expected exact totals including the three adjustments are shown below. Float totals can differ in low-order digits; the exact difference must be measured by running the benchmark rather than guessed.

Base payments N	Exact total including 3 × ■0.10	Exact Decimal total
300	300 × ■33.40 + ■0.30	■10,020.30
6,000	6,000 × ■33.40 + ■0.30	■200,400.30
150,000	150,000 × ■33.40 + ■0.30	■5,010,000.30

```
from decimal import Decimal
from time import perf_counter

def benchmark(n):
```

```
    amounts = ["33.40"] * n + ["0.10"] * 3
    start = perf_counter()
    f_total = sum(float(x) for x in amounts)
    float_seconds = perf_counter() - start
    start = perf_counter()
    d_total = sum((Decimal(x) for x in amounts), Decimal("0.00"))
    decimal_seconds = perf_counter() - start
    print(n, f_total, d_total, Decimal(str(f_total)) - d_total,
          float_seconds, decimal_seconds)

for n in (300, 6000, 150000):
    benchmark(n)
```

Rounding each farmer's payment is the recommended ledger rule because the bank pays each farmer in paisa; then sum the rounded payment records. Rounding only the final aggregate may differ if intermediate amounts have more than two decimals. This implementation rounds rate and payment to two decimal places; if rates or payments can have sub-paisa precision, confirm whether the accounting policy rounds the rate, the payment, or both.

Alternative: cache a computed rate for each distinct (fat, snf) pair. A dictionary lookup is average O(1), compared with a small fixed number of arithmetic operations also O(1) per entry. For n records and k distinct quality pairs, cache space is O(k); it helps most when k is much smaller than n. Benchmark with time.perf_counter() or timeit; timings are machine-dependent and must be copied from a real execution.

Task 11 – Recommend and defend the final design (4 marks)

• Use Decimal constructed from input strings for litres, percentages, rates and amounts; never construct Decimal from float for financial inputs.

• Keep a single rate card with named constants; seasonal rates should be data/configuration rather than scattered literals.

• Validate before calculation and round the farmer's payable amount to two decimal places using an explicitly agreed policy (this report uses ROUND_HALF_UP).

• Use pure calculation functions, a separate validation layer, a module and automated assertions; retain a transaction ledger for audit/reconciliation.

• At 150,000 records, O(n) processing is suitable. Use streaming/running totals for low memory, or a database/batch ledger when audit, correction and reporting requirements justify persistent storage.

Limitations: the SNF validation range is assumed and needs approval; no taxes/deductions, rejected milk, duplicate IDs, analyser calibration or transaction database rules were supplied. The rate formula's step rounding for two-decimal readings is an explicit assumption. District-scale use should add persistent storage, transaction IDs, role-based access, backups, logging, reconciliation and a documented rounding policy.

Rate revision process: update the central rate-card configuration and effective dates, run regression tests for each season and boundaries, review outputs with the treasurer, version the rate card, and deploy only after approval. The rate calculation itself remains O(1) per entry, so 150,000 entries remain O(n) overall.

Task 12 – Re-evaluate and adapt the design (8 marks)

A. Impact analysis (2 marks)

New requirements: analyser fat/SNF readings have two decimal places; April–June base rate is ■32.00; fat step value becomes ■0.45; and farmers supplying more than 500 litres during the ten-day period receive ■0.50 per litre bonus. The reference values and SNF step value were not stated as changed, so this design retains fat reference 3.5%, SNF reference 8.5%, step size 0.10 percentage points and SNF value ■0.30, while making the assumptions explicit.

Two-decimal readings mean a quality value may be 3.56%, for example. The formula must define whether steps are rounded to the nearest 0.10 percentage point; this design uses Decimal ROUND_HALF_UP on (reading – reference) / 0.10. If the analyser's values are intended to be prorated rather than step-rounded, the co-operative must change the rule.

B. Evaluate the original design (2 marks)

Named constants and pure functions remain suitable, but the original single fixed rate card must be modified to select a card by season. The payment function must also receive the farmer's aggregate ten-day litres to determine bonus eligibility; a per-entry-only function cannot correctly decide a ten-day threshold unless it has access to the aggregate. The module structure should be retained and extended.

C. Adapted design and code (2 marks)

```
from decimal import Decimal, ROUND_HALF_UP
```

```
D = Decimal
STEP = D("0.10")
FAT_REF = D("3.50")
SNF_REF = D("8.50")
SNF_VALUE = D("0.30")
STANDARD = {"base": D("30.00"), "fat_value": D("0.40")}
APR_JUN = {"base": D("32.00"), "fat_value": D("0.45")}
BONUS_THRESHOLD = D("500")
BONUS_PER_LITRE = D("0.50")

def steps(reading, reference):
    """Return nearest 0.10-percentage-point step, with half-up ties."""
    return ((reading - reference) / STEP).quantize(
        D("1"), rounding=ROUND_HALF_UP)

def seasonal_rate(fat, snf, season):
    """Return price per litre before volume bonus for a named season."""
    card = APR_JUN if season == "APR_JUN" else STANDARD
    rate = (card["base"] + steps(fat, FAT_REF) * card["fat_value"]
            + steps(snf, SNF_REF) * SNF_VALUE)
    return rate.quantize(D("0.01"), rounding=ROUND_HALF_UP)

def ten_day_payment(litres, fat, snf, season, ten_day_litres):
    """Return payment; bonus applies only if ten-day volume exceeds 500 L."""
    rate = seasonal_rate(fat, snf, season)
    bonus = BONUS_PER_LITRE if ten_day_litres > BONUS_THRESHOLD else D("0.00")
    return (litres * (rate + bonus)).quantize(
        D("0.01"), rounding=ROUND_HALF_UP)

# Demonstrations
print(seasonal_rate(D("3.56"), D("8.50"), "STANDARD"))
print(seasonal_rate(D("3.56"), D("8.50"), "APR_JUN"))
print(ten_day_payment(D("520"), D("3.50"), D("8.50"),
                      "APR_JUN", D("520")))
print(ten_day_payment(D("500"), D("3.50"), D("8.50"),
                      "APR_JUN", D("500")))
```

Expected demonstration outputs:

```
30.40
32.45
16900.00
16000.00
```

Explanation: 3.56% is 0.06 above 3.50%; nearest-step rounding yields one 0.10 step under ROUND_HALF_UP, so the standard rate is 30.00 + 0.40 = 30.40 and April–June is 32.00 + 0.45 = 32.45. For 520 L in April–June at reference quality, rate plus bonus is ■32.50/L and payment is ■16,900.00. At exactly 500 L, no bonus applies, giving ■16,000.00. The brief says more than 500 litres, not 500 or more.

D. Evidence-based defence (2 marks)

Test	Season / quality / volume	Expected
C1	Standard, fat 3.56, SNF 8.50	■30.40/L
C2	April–June, fat 3.56, SNF 8.50	■32.45/L
C3	April–June, 520 L at reference quality	■16,900.00 including bonus
C4	April–June, exactly 500 L at reference quality	■16,000.00; no bonus
C5	April–June, 501 L at reference quality	■16,275.50 including bonus

Decision: MODIFY, not redesign. The core concepts—Decimal, named constants, validation, reusable functions and module separation—remain appropriate. Modify the rate card to be season-aware and compute the bonus from the ten-day aggregate. Trade-offs: configuration and testing become more complex, but the change is localised and avoids duplicating rate formulas. Run the tests on the target machine and include the actual console output as evidence.

Appendix A – Complete reference implementation

Save the following as milk_calculator.py. It is a compact runnable implementation for the original requirements with validation, a running ten-day total, and tests. The Task 12 seasonal code is shown above.

```
from decimal import Decimal, ROUND_HALF_UP, InvalidOperation

D = Decimal
CENT = D("0.01")
ONE = D("1")
BASE_RATE = D("30.00")
FAT_REF = D("3.5")
```

```
SNF_REF = D("8.5")
STEP = D("0.1")
FAT_VALUE = D("0.40")
SNF_VALUE = D("0.30")
MIN_FAT = D("2.0")
MAX_FAT = D("7.0")
# Assumption: confirm this SNF range with the co-operative.
MIN_SNF = D("6.0")
MAX_SNF = D("10.0")

def parse_decimal(text):
    """Parse a decimal from text without introducing binary-float error."""
    try:
        return D(text.strip())
    except (InvalidOperation, AttributeError):
        raise ValueError(f"Invalid decimal input: {text!r}")

def validate_entry(farmer_id, litres, fat, snf, shift):
    """Raise ValueError when an entry violates the collection rules."""
    if not farmer_id or not farmer_id.strip():
        raise ValueError("Farmer ID is required")
    if litres <= 0:
        raise ValueError("Litres must be greater than zero")
    if not (MIN_FAT <= fat <= MAX_FAT):
        raise ValueError("Fat must be between 2.0 and 7.0")
    if not (MIN_SNF <= snf <= MAX_SNF):
        raise ValueError("SNF must be between 6.0 and 10.0")
    if shift.upper() not in {"MORNING", "EVENING"}:
        raise ValueError("Shift must be MORNING or EVENING")

def step_count(value, reference):
    """Return the nearest whole 0.1 percentage-point step."""
    return ((value - reference) / STEP).quantize(
        ONE, rounding=ROUND_HALF_UP)

def rate_per_litre(fat, snf):
    """Calculate and return the rounded price per litre."""
    rate = (BASE_RATE + step_count(fat, FAT_REF) * FAT_VALUE
            + step_count(snf, SNF_REF) * SNF_VALUE)
    return rate.quantize(CENT, rounding=ROUND_HALF_UP)

def payout(litres, fat, snf):
    """Return one payment rounded to the nearest paisa."""
    return (litres * rate_per_litre(fat, snf)).quantize(
        CENT, rounding=ROUND_HALF_UP)

def ten_day_total(payments):
    """Sum already-rounded payment amounts."""
    return sum(payments, D("0.00")).quantize(CENT)

def run_tests():
    """Run deterministic assertions for reference and boundary cases."""
    assert rate_per_litre(D("3.5"), D("8.5")) == D("30.00")
    assert payout(D("10"), D("3.5"), D("8.5")) == D("300.00")
    assert payout(D("8.5"), D("4.2"), D("8.7")) == D("283.90")
    assert payout(D("12"), D("3.1"), D("8.3")) == D("333.60")
    assert payout(D("1"), D("2.0"), D("8.5")) == D("24.00")
    assert payout(D("1"), D("7.0"), D("8.5")) == D("44.00")
    assert ten_day_total([D("300.00"), D("283.90"),
                          D("333.60"), D("24.00"), D("44.00")]) == D("985.50")
    try:
        validate_entry("F001", D("-1"), D("3.5"), D("8.5"), "MORNING")
    except ValueError:
        pass
    else:
        raise AssertionError("Negative litres should be rejected")
    try:
        validate_entry("F002", D("1"), D("7.1"), D("8.5"), "EVENING")
    except ValueError:
        pass
    else:
        raise AssertionError("Fat above range should be rejected")
    print("All tests passed.")

if __name__ == "__main__":
    run_tests()
    print("Reference payment:", payout(D("8.5"), D("4.2"), D("8.7")))
```

Appendix B – Suggested GitHub repository structure

```
CSA08-1919260011-Assignment1/
■■■ README.md
■■■ requirements.txt
■■■ Task01/
■   ■■■ analysis.md
■■■ Task02/
■   ■■■ interpreter_session.txt
■   ■■■ decimal_total.py
■■■ Task03/
■   ■■■ constants.py
```

```
■■■ Task04/
■   ■■■ expressions.py
■■■ Task05/
■   ■■■ input_validation.py
■■■ Task06/
■   ■■■ payment_functions.py
■■■ Task07/
■   ■■■ milkrate.py
■   ■■■ test_milkrate.py
■■■ Task08/
■   ■■■ pseudocode.txt
■   ■■■ flowchart.png
■■■ Task09/
■   ■■■ milk_calculator.py
■   ■■■ test_results.txt
■■■ Task10/
■   ■■■ benchmark.py
■■■ Task11/
■   ■■■ recommendation.md
■■■ Task12/
    ■■■ seasonal_milkrate.py
    ■■■ challenge_tests.txt
```

Suggested README.md details

```
# CSA08-1919260011-Assignment1
Name: S Jai Aakash
Register number: 1919260011
Python: Record the installed version using `python --version`
Libraries: Standard library only (decimal, time, sys)
Install: No third-party dependencies required
Run original calculator: `python milk_calculator.py`
Run tests: `python milk_calculator.py`
Run module demonstration: `python milkrate.py`
Run benchmark: `python benchmark.py`
Repository URL: Add the actual GitHub URL after creating the repository.

For each task, include the source, test inputs, captured console output
and a short result note. Commit each task separately as work progresses.
```

Final submission checklist

• Run every program in the chosen Python version and save actual output screenshots.

• Run the benchmark for 300, 6,000 and 150,000 payments; paste measured times and observed float-versus-Decimal differences into Task 10.

• Confirm the accepted SNF range and the official rounding policy with the course instructor/co-operative.

• Create the GitHub repository, commit work progressively, and replace the placeholder with the actual repository URL.

• Review the PDF, ensure the cover page and report carry the same identity details, and prepare to explain the code in the viva.